

3. Voraussetzungen

4. Quelltext

5. Pakete, Klassen, Variablen

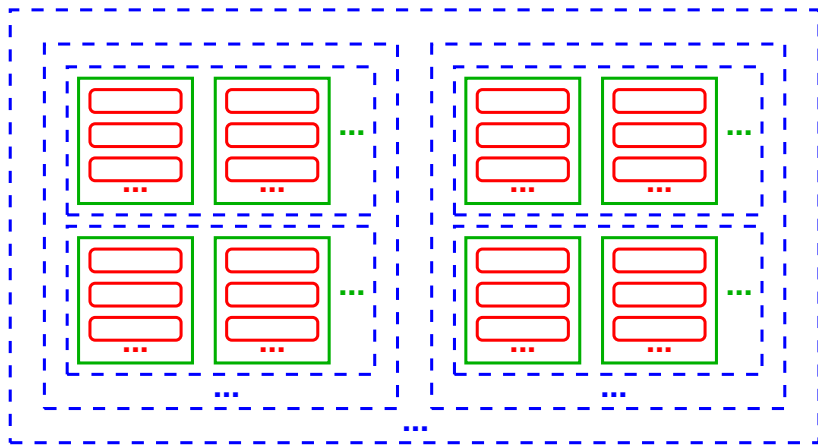
- Pakete
- Klassen und Objekte
- Variablen
- Referenzen

6. Methoden

Pakete

Strukturierung von Quellcode

in großen SW-Projekten: **Klassen** in **Paketen** in **Paketen** in ...



Pakete

Paket: reine **Sammlung** (ohne Eigenschaften) von Klassen/Paketen

Pakete modellieren **Themenbereiche** bzw. **-unterbereiche**

Pakete, Klassen, Methoden bilden **Komponenten** (**Bausteine**):

- in sich **abgeschlossen** („gekapselt“)
- voneinander klar **getrennt**, jeweils **eindeutiger Zweck**
- möglichst **allgemein** formuliert
- zu beliebig großen Systemen **kombinierbar**

→ **Softwarearchitektur**

Hinweis: Pakete i.d.R. **unnötig im Rahmen dieser Veranstaltung**
(Strukturierungsaufwand für kleine Programme unangemessen)

Pakete: Deklaration

package-Deklaration am **Dateianfang** weist Dateiinhalt Paket zu:

```
package biologie;  
public class Pilz { /*...*/ }
```

qualifizierter Name der Klasse ist nun `biologie.Pilz`

Konventionen:

- Paketbezeichner werden **klein** geschrieben
- **Pakete** $\hat{=}$ **Verzeichnis** (`Pilz.java` in `biologie`)

gleiches Muster bei **geschachtelten Paketen**:

```
package wissenschaft.biologie.botanik;  
public class Pflanze { /*...*/ }
```

Klasse `wissenschaft.biologie.botanik.Pflanze` ist deklariert in
Datei `wissenschaft/biologie/botanik/Pflanze.java`

Verwendung in anderen Paketen

Zugriff auf Paketelemente per qualifiziertem Namen:

```
java.util.Scanner sc = new java.util.Scanner(System.in);
```

Alternative: **import** zur Vermeidung von Wiederholungen:

```
import java.util.Scanner;           // macht 'Scanner' bekannt
...
Scanner sc = new Scanner(System.in);
```

Alternative: **Wildcard** für alle Elemente eines Pakets:

```
import java.util.*;                // macht alle Paketelemente bekannt
...
Scanner sc = new Scanner(System.in);
Random zufallGenerator = new Random(); // in java.util
int zufallWert = zufallGenerator.nextInt();
```

Hinweis: **java.lang.*** (z.B. System) wird automatisch importiert

Verwendungsbeispiel

```
1 package mathe;  
2 public class Geometrie {  
    // ...  
7 }
```

mathe/Geometrie.java

```
1 package test;  
2 import mathe.*; // ausser Geometrie auch Mathe  
3 public class GeometrieTest {  
    // ...  
13 }
```

test/GeometrieTest.java

```
$ javac mathe/Geometrie.java mathe/Mathe.java
```

```
$ javac test/GeometrieTest.java
```

```
$ java test.GeometrieTest ← main der Klasse in Paket ausführen
```

Umfang eines Rechtecks ...

Klassen und Objekte

Klassen: Deklaration

Java-Code ist (innerhalb von Paketen) in **Klassen** gegliedert

Eine **Klassendeklaration** hat die Form:

KlassenDeklaration:

KlassenKopf KlassenKoerper

KlassenKopf:

[public] [...] class KlassenBez [...]

KlassenKoerper:

{ { Element } }

(darin bezeichnen ... Bestandteile, die später vorgestellt werden)

Konventionen (Wiederholung):

- *KlassenBez*: erst **Großbuchstabe**, dann **CamelCase**
- Klassen werden i.d.R. **Javadoc-Klassenkommentare** vorangestellt

Klassen: Zugriffsrecht

Einschränkung des Zugriffs auf Klasse durch **Zugriffsmodifikator**:

public (öffentlicher Zugriff): Klasse überall verwendbar

kein Mod. (Paketzugriff): Klasse nur aus selbem Paket verwendbar

- im Paket Zugriff auf „Hilfs“klassen für gem. interne Aufgaben
- außerhalb kein Zugriff (**Kapselung**: Schutz von Interna)

(nur) **public**-Klassen müssen in gleichnamiger Datei stehen,
in selber Datei andersnamige Klassen mit Paketzugriff möglich

Hinweis: Paketzugriffsrecht in dieser Veranstaltung nicht verwendet
(erst in größeren Projekten interessant)

Objekte

bisher bekannt: Klassen

als **allgemeine, statische** Modelle von **Begriffen**

→ **Eigenschaften** gelten **universell**

außerdem: Objekte (Instanzen von Klassen)

als **spezielle, dynamische** Modelle von **Exemplaren** einer Klasse

→ **Eigenschaften** gelten **individuell**

Beispiel:

Klasse: Mensch (Säugetier, ca. 8.4 Mrd. Objekte, 206 Knochen, ...)

Objekte:

- "Monika", 1992 geboren, 169 cm groß, ...
- "Markus", 1998 geboren, 178 cm groß, ...
- ...

Objekte: Instanziierung & Lebensdauer

Instanziieren einer Klasse (**Erzeugen** eines Objektes) durch **new**:

```
new KlassenBez()
```

liefert eine **Referenz** (**Verweis**, später mehr) auf ein neues Objekt

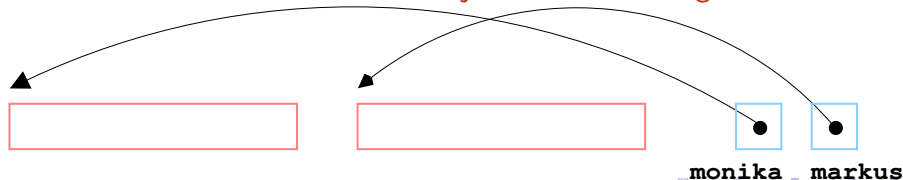
Typ der **Referenz** und der **Instanz** ist der **Klassentyp**

Beispiel:

```
Mensch monika = new Mensch();
```

```
Mensch markus = new Mensch();
```

→ Referenzen auf **verschiedene Objekte** mit **indiv. Eigenschaften**



Objekte: Lebensdauer

beliebig viele **Objekte** einer Klasse können erzeugt werden;
Lebensdauer ist so lang wie das Objekt **referenziert** wird
die **Klasse** selbst existiert **genau einmal** pro **Programmausführung**

Elemente

als *Element* (**member**) einer Klasse bisher: **Klassenmethode**
weitere Elemente werden im Folgenden vorgestellt

Deklaration meist nach dem Muster:

Element :

Zugriff [static] [...] *Typ* *ElementBez* *ElementSpez*

Deklaration **mit static**: **Klassenelement**

Deklaration **ohne static**: **Instanzelement**

Beispiel: eine **Instanzmethode** wird wie eine Klassenmethode deklariert, aber ohne static:

```
public class Mensch {  
    ...  
    public int geburtsjahr() {  
        ...  
    }  
}
```

Elemente: Bezeichner

Bezeichner *ElementBez* ist **überall in Klasse** bekannt/verwendbar, unabhängig von der Reihenfolge der Deklarationen

Hinweis: in Dokumentation etc. benennt man zur Unterscheidung

- Klassenlement als *KlassenBez.ElementBez*
- Instanzelement als *KlassenBez#ElementBez*

Elemente: interner Zugriff

innerhalb der Klasse prinzipiell **direkter Zugriff** auf das Element:

ElementBez

Beispiel:

```
public class Mensch {  
    ...  
    public static int anzahl() {  
        return 8_400_000_000; // eigentlich zu gross für int  
    }  
    public int geburtsjahr() {  
        ...  
    }  
    public int alter(int j) {  
        return j - geburtsjahr(); // interner Zugriff  
    }  
}
```


Elemente: externer Zugriff

außerhalb der Klasse **qualifizierter Zugriff** auf

- **Klassenelement über Klasse oder Referenz:**

KlassenBez.ElementBez

Referenz.ElementBez

Beispiel:

```
Mensch m = new Mensch();  
... Mensch.anzahl() ...;  
... m.anzahl() ...;           // selbe Methode
```

- **Instanzelement nur über Referenz:**

Referenz.ElementBez

Beispiel:

```
... m.geburtsjahr() ...;
```

Elemente: Abhängigkeiten

Elemente können in folgender Weise aufeinander zugreifen:

- Instanzelemente auf Klassenelemente
(„Objekte kennen Klasse“)

Beispiel: Instanzmethode `Mensch#istAlt()` kann
Klassenmethode `Mensch.alterDurchschnitt()` aufrufen

- Klassenelemente **nicht** auf Instanzelemente
(„Klasse kennt Objekte **nicht**“)

Beispiel: Klassenmethode `Mensch.alterDurchschnitt()`
kann Instanzmethode `Mensch#alter()` nicht aufrufen

Elemente: Zugriffsrecht

Element :

Zugriff [static] [...] *Typ ElementBez ElementSpez*

Modifikator *Zugriff* gibt an, von wo *Zugriff* erlaubt ist:

private: nur aus selber Klasse

<leer>: zudem aus selbem Paket

protected: (selten) zudem aus abgeleiteten Klassen (dazu später)

public: von überall, wo *Zugriff* auf Klasse

Hinweis: in diesem Kurs nur **public** und **private** relevant

bei Anwendung von **Zugriffsmodifikator** auf Klasse

entsprechende **Limitierung des Zugriffs** auf ihre **Elemente**

Beispiel: public-Element in Paket-sichtb. Klasse → Paket-sichtbar

Elemente: Zugriffsrecht `private`

`private` für Elemente, die von Klasse/Objekten nur `intern` (selbst) verwendet werden, z.B. für Klasse `Mensch` Element `kontostand`

wichtig: `private` heißt nicht „privat“:

alle Objekte einer Klasse haben untereinander `privaten Zugriff`!

z.B. hat `monika` Zugriff auf den `kontostand` von `markus`
(unkritisch, weil in `Mensch` festgelegt ist, was `monika` mit der Information machen kann/darf)

Hinweis: objekt-privater Zugriff in Java nicht direkt darstellbar!

Variablen

Variablen

Variable beschreibt

- technisch: **Speicherplatz** mit **Typ** und **Name** (per **Deklaration**)
- modellierend: **Zustand**, der sich über die Zeit ändern kann

bisher bekannt: **lokale Variable** in Methode:

```
LokaleVariablenDeklaration:  
Typ VariablenBez [ = Ausdruck ] ;
```

Konvention (Wiederholung):

VariablenBez: erst **Kleinbuchstabe**, dann **CamelCase**

Unterschied zu **Methode**: keine Parameter-/Argumentliste (...)

Variablen: Verwendung

Wiederholung:

Lesen eines Wertes aus Speicherplatz:

VariablenBez als *Ausdruck* für aktuellen **Wert der Variablen**

Schreiben eines Wertes in Speicherplatz (**Zuweisung**):

ZuweisungsAusdruck:

SpeicherplatzAusdruck = *Ausdruck*

VariablenBez ist ein *SpeicherplatzAusdruck*

Ausdruck muss **kompatiblen Typ** haben

erst wird *Ausdruck* **ausgewertet**, dann Wert **zugewiesen**:

```
x = 6 * x + 4;
```

nach Zuweisung ist der **alte Wert überschrieben** (d.h. verloren)

Klassen- und Instanzvariablen

Variable als Element einer Klasse:

VariablenDeklaration:

Zugriff `[static] [...]` *Typ* *VariablenBez* `[= Ausdruck]` ;

beschreibt **Zustand** von **Klasse** oder **Objekt**

Konventionen:

- *Zugriff* auf Variable sollte **immer(!) private** sein
- in Klasse **erst Variablen, dann Methoden** deklarieren

Klassen- und Instanzvariablen: Initialisierung

VariablenDeklaration:

Zugriff [static] [...] *Typ VariablenBez* [= Ausdruck] ;

optional **Initialisierung** mit *Ausdruck* als **Startwert**

falls nicht gegeben: typabhängige **automatische Initialisierung**:

int:	0	boolean:	false
double:	0.0	Referenz:	null (dazu später)

Kontrast: **lokale Variable** wird **nicht automatisch initialisiert**

Klassen- vs. Instanzvariablen

Variablendeklaration

- mit `static`: **Klassenvariable**
⇒ existiert je einmal pro Klasse
- ohne `static`: **Instanzvariable** (oder **Attribut**)
⇒ existiert je einmal pro Objekt

Klasse/Objekt ohne Klassen-/Instanzvariablen: **zustandsfrei** („leer“)

Lebensdauer:

- Klassenvariable: so lange wie jeweilige Klasse
d.h. **während gesamter Programmausführung**
- Instanzvariable: so lange wie jeweiliges Objekt

Klassenvariable: Beispiel

```
1 public class KontaktMitSprache {  
2     private static String sprache = "Englisch";  
3     public static void setzeSprache(String s) {  
4         sprache = s;  
5     }  
6     public static String gruss(String name) {  
7         if (sprache.equals("Deutsch")) {  
8             return "Hallo " + name + "!";  
9         } else {  
10            return "Hello " + name + "!";  
11        }  
12    }  
13 }
```

```
KontaktMitSprache.gruss("Bruce"); // liefert "Hello Bruce!"  
KontaktMitSprache.setzeSprache("Deutsch");  
KontaktMitSprache.gruss("Bruce"); // liefert "Hallo Bruce!"
```

Klassenvariable: Beispiel

häufige Verwendungen: **Einstellungen** (z.B. sprache) oder **Zähler**

```
public class KontaktMitZaehler {  
    // ...  
    private static int grussZaehler = 0;  
    public static String gruss(String name) {  
        ++grussZaehler;  
        // ...  
    }  
    // ..  
}
```

Klassen- vs. Instanzvariablen: Beispiel

Instanzvariablen modellieren individuelle Eigenschaften

Beispiel:

```
1 public class Mensch {  
2     private static int a; // Anzahl  
3     private String n;    // Name  
4     private int j;       // Geburtsjahr  
    // ...  
33 }
```

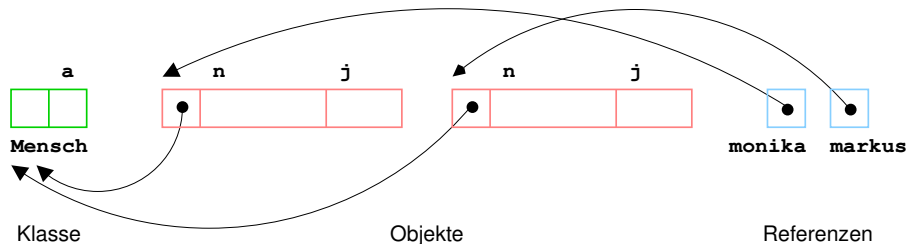
Klassenvariablen: Eigenschaften der Menschheit
(weitere z.B.: max. Alter, Anzahl Knochen, ...)

Instanzvariablen: Eigenschaften der einzelnen Menschen
(weitere z.B.: Größe, Gewicht, Adresse, Kontostand, ...)

Klassen- vs. Instanzvariablen: Darstellung

```
Mensch mo = new Mensch();  
// Setzen der Klassen- und Instanzvariablen von 'mo'  
Mensch ma = new Mensch();  
// Setzen der Klassen- und Instanzvariablen von 'ma'
```

im Speicher sind Klasse und Objekte (ungefähr) abgelegt als:



Klassen- vs. Instanzvariablen: Zugriff

Hinweis: Beispiele hier nur zur Veranschaulichung –
gezeigter Code nur in Klasse selbst möglich,
wenn Variablen als `private` deklariert sind

Zugriff auf Instanzvariable nur über Referenz:

```
mo.n = "Monika";  
mo.j = 1992;
```

Zugriff auf Klassenvariable über Klasse oder Referenz:

```
... Mensch.a == mo.a ... // immer true: selbe Variable
```

dabei über alle Referenzen Zugriff auf selbe Klassenvariable:

```
... mo.a == ma.a ... // immer true: selbe Variable
```

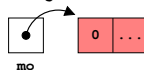
aus Klassenmethode kein direkter Zugriff auf Instanzvariable

Referenzen

Referenzen

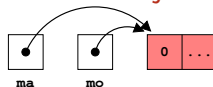
Referenzen: Verweise auf Objekte im Heap
klein im Vergleich zu (ggf. sehr großen) Objekten

```
Mensch mo = new Mensch();
```

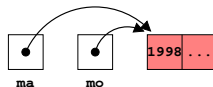


verschiedene Referenzen können auf **selbes Objekt** verweisen

```
Mensch ma = mo;
```



```
ma.j = 1998;
```



```
// mo.j == 1998
```

Änderungen an einer Referenz **auch über andere Referenzen sichtbar!**

Operationen

Zuweisung und **Vergleich** sind **einzigste Operationen** auf Referenzen
(dazu gleich mehr)

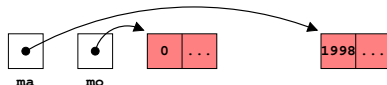
Zugriff wie `mo.geburtsjahr()` wirkt nicht auf Referenz `mo`,
sondern **auf das referenzierte Objekt**

Hinweis:

„Zuweisung“ auch **Übergabe** als Argument in **Methodenaufruf**

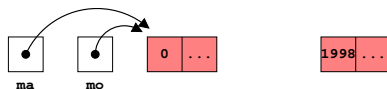
Freigabe von Speicher

```
mo = new Mensch();
```



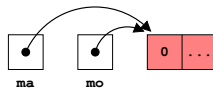
bei **Zuweisung an einzige Referenz** auf ein Objekt

```
ma = mo;
```



geht nicht mehr referenziertes Objekt verloren!

Garbage Collector (Teil der JVM) gibt verlorenen Speicher frei



Vergleich

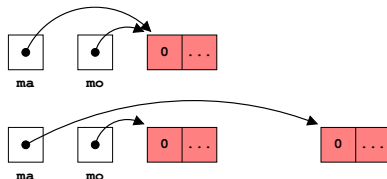
Vergleich von Referenzen vergleicht **Adressen**, nicht Speicherinhalt:

```
// ma == mo
```

```
ma = new Mensch();
```

```
// ma != mo
```

ma == mo prüft **Identität** (nicht bloß **Gleichheit**!) der Objekte



Vergleich von String-Referenzen

darum Vergleich von Strings mit Instanzmethode `equals`:

```
String s1 = "Hallo";  
String s2 = s1;  
String s3 = "Hallo";  
// s1 == s2           // selbe String-Instanz  
// s1.equals(s2)      // somit wertgleich  
  
// s1 != s3           // verschiedene Instanzen (aber s.u.!)  
// s1.equals(s3)      // jedoch auch wertgleich
```

Besonderheit für **Zeichenketten**:

eigentlich: `s1 != s3` (**verschiedene Referenzen**)

aber: **Zeichenketten** sind **unveränderlich**, darum darf Compiler zur **Optimierung** beide ZK "Hallo" durch **selbe Instanz** darstellen

→ **ungewiss**, ob `s1 != s3` ⇒ **immer equals verwenden!**

Referenztypen

Referenztyp: Werte sind **Referenzen**, nicht die Objekte selbst

Klassentypen sind **Referenztypen**

für Objekte von Referenztypen gilt:

- sind selbst **anonym**, nur über Referenzen benannt/erreichbar
- werden durch **new** erzeugt
- werden mit **equals** verglichen, **==** prüft **Identität**
- werden **automatisch** aus Speicher **gelöscht**, sobald keine Referenz mehr darauf verweist (**Garbage Collection**)

spezielle Referenz: null

null ist spezieller Referenzwert (kein Objekt referenziert):

- Default-Wert bei automatischer Initialisierung
- zur Markierung von Referenzen als „nicht gesetzt“

```
if (mo == null) {  
    ...  
}
```

- zur expliziten Freigabe von nicht mehr benötigtem Objekt

```
mo = null; // Garbage Coll. kann Objekt entfernen
```

- als Rückgabewert von Methoden, falls kein Ergebnis definiert
- Zugriff auf null-Referenz verursacht **NullPointerException**:

```
... mo.geburtsjahr() ...
```

- aber Umwandlung in String legal: "" + null ergibt "null"